DATA-260 Homework 10 — Pipeline Evaluation, Judge Validation, Distributed Tracing, Operations Dashboard

Xu Wang_018319275

GitHub repository: https://github.com/xuwang-goldenwater/data260-9275 (code: code/observability/, code/eval_pipeline/, code/ops_dashboard/; evidence: reports/hw10/)

Built in the order 4 → 2 → 3 → 1: the evaluator writes spans (Part 4), the 20 judge-validation drafts come from the 50-request latency run (Part 4.3), and the dashboard reads all of them. The sections below follow the assignment order.

Section 0 — Personal Configuration

Value	Definition	My value
SID4	Last four digits of SJSU ID	9275
PORT_BASE	8000 + (SID4 mod 900)	8275
PREFIX	"s" + SID4	s9275 (topics s9275_*, Redis keys s9275:*, group s9275_evaluator)
SEED	SID4	9275
VERIFY_SEED	260000 + SID4	269275
DOMAIN_ID	SID4 mod 8	3 — Grocery supply and recall notices

Item	Value
Hardware	MacBook Pro, Apple M1, 8 GB RAM; Docker Desktop
Kafka	apache/kafka 3.8 (KRaft); topics s9275_inbox / _tasks (2 partitions) / _drafts / _final / _dead_letter
Cache / DB	Redis Stack 7.4 semantic cache (MiniLM, cosine, threshold 0.85 + recall guard); MySQL 8.0 s9275_rel
Agents	HW8 Planner → Writer → Reviewer, qwen2.5:3b via Ollama (temperature 0, seed 9275)
Judge	qwen3:4b via Ollama, local; DeepEval 4.3.4 GEval; temperature 0, seed 9275, num_ctx 4096, num_predict 300, thinking off, JSON-schema output, 90 s timeout; scale 1–5
Dashboard	Streamlit 1.67 (code/ops_dashboard/app.py)
Builds on	HW8 Kafka pipeline and correlation_id; HW9 Redis semantic cache (hit / miss)

Part 1 — Streamlit Operations Dashboard

`streamlit run code/ops_dashboard/app.py` (make hw10-dashboard). The dashboard is an evidence viewer for the existing system: every number is read from files the system wrote (Parts 2–4), and every metric states its source file and time window under it.

Page	What it shows	Data source
Summary (1.1)	Requests; cache hit rate with hits and misses shown separately; error rate (exceptions / timeouts); Reviewer rejection rate (an outcome, not an error); end-to-end latency p50 / p95 for cache misses and, separately, cache	raw/spans.jsonl (gateway spans), live Redis, raw/latency_summary.json

Page	What it shows	Data source
	hits; live cache size; Part 4.3 stage table; recent requests. Time window selectable (all / 1 h / 24 h / 7 d).	
Evaluation (1.2)	Part 2 scores by stage (plan, draft, final) and by question kind; per-case table; final vs draft; coordination interpretation. Part 3 Spearman, MAD, repeatability, per-kind gap, raw per-case scores, verdict.	raw/eval_results.jsonl, part3/judge_validation.json, part3/judge_runs.csv
Trace lookup (1.3)	trace_id input (full id or prefix) and buttons for the three example traces; header marks CACHE HIT / CACHE MISS / FAILED (with failing stage); every span in stage order with start, end, duration, status, error; timeline chart; the trace's evaluation scores.	raw/spans.jsonl, raw/trace_examples.json, raw/eval_results.jsonl

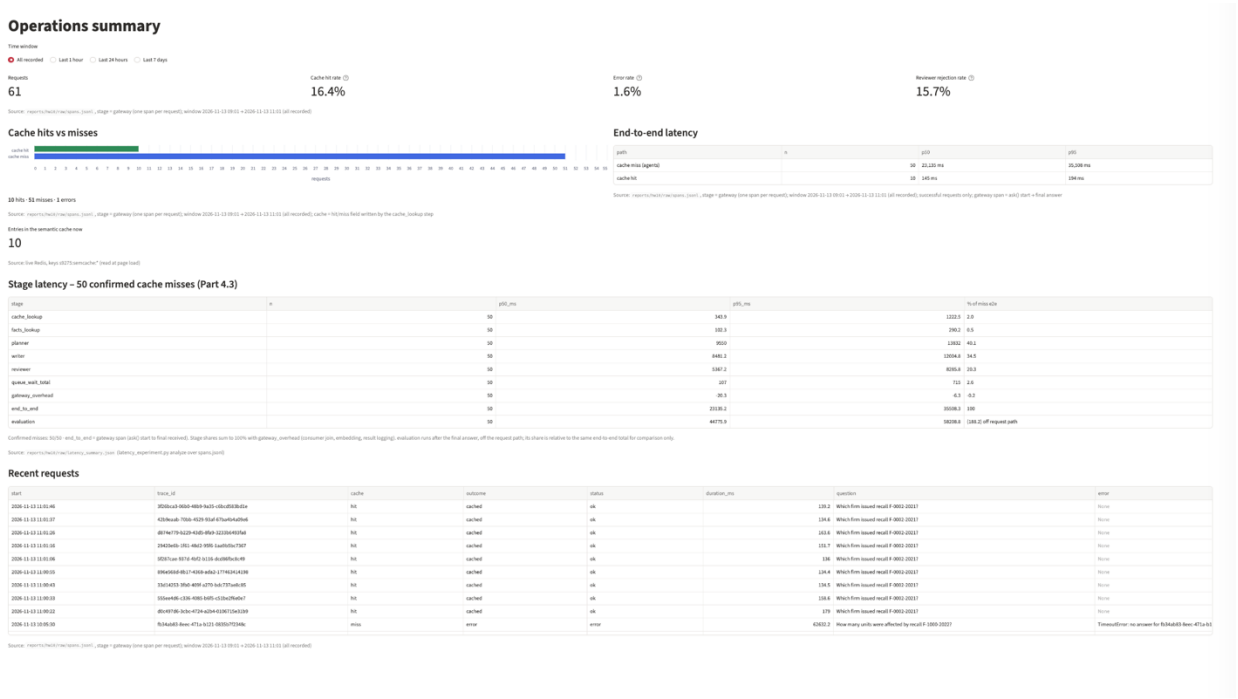
code/ops_dashboard/app.py — caching

```
@st.cache_data(ttl=30, show_spinner=False)
def read_jsonl(path: str, mtime: float) -> list[dict]:
    """Cached per file version: re-read only when the file changes (or after 30 s)."""
    p = Path(path)
    if not p.exists():
        return []
    return [json.loads(line) for line in p.read_text().splitlines() if line.strip()]


@st.cache_resource(show_spinner=False)
def redis_client():
    """One Redis connection for the whole app (a reusable resource, not data)."""
    import redis
    r = redis.Redis.from_url(os.getenv("REDIS_URL", "redis://localhost:6379/0"),
                             decode_responses=True, socket_connect_timeout=1)
    r.ping()
    return r
```

`@st.cache_data` caches each file read, keyed on the file's modification time (so a new span appears after at most 30 s); `@st.cache_resource` keeps one Redis connection for the whole app, because a connection is a reusable resource, not data.

Summary over all recorded requests (2026-11-13 09:01–11:01): 61 requests = 50 latency misses + 1 failed demo + 10 cache hits. Hit rate 16.4% (10/61) — low by design, because the 50 latency requests were chosen to miss; error rate 1.6% (1/61, the injected failure); Reviewer rejection rate 15.7% (8/51 misses). Cache-miss end-to-end p50 23.1 s; cache-hit p50 145 ms (n = 10).



Operations summary

Time window

All recorded | Last 1 hour | Last 24 hours | Last 7 days

Requests	Cache hit rate	Error rate	Reviewer rejection rate
61	16.4%	1.6%	15.7%

Source: reports/hw3/raw/spans.jsonl , stage = gateway (one span per request); window 2026-11-13 09:01 → 2026-11-13 11:01 (all recorded)

Cache hits vs misses

cache hit
cache miss

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55

requests

10 hits · 51 misses · 1 errors

Source: reports/hw3/raw/spans.jsonl , stage = gateway (one span per request); window 2026-11-13 09:01 → 2026-11-13 11:01 (all recorded); cache = hit/miss field written by the cache_lookup step

Entries in the semantic cache now

10

Source: live Redis, keys cl1275:semcache:* (read at page load)

End-to-end latency

path	n	p50	p90
cache miss (agent)	10	21,135 ms	25,206 ms
cache hit	10	145 ms	194 ms

Source: reports/hw3/raw/spans.jsonl , stage = gateway (one span per request); window 2026-11-13 09:01 → 2026-11-13 11:01 (all recorded); successful requests only; gateway span = ask() start → final answer

Stage latency – 50 confirmed cache misses (Part 4.3)

stage	n	p50_ms	p95_ms	% of miss e2e
cache_lookup	50	343.9	1222.5	2.0
facts_lookup	50	102.3	290.2	0.5
planner	50	9550	13832	40.1
writer	50	8481.2	12004.8	34.5
reviewer	50	3347.2	8295.0	10.3
queue_wait_total	50	107	710	2.6
gateway_overhead	50	30.5	41.3	0.2
end_to_end	50	23130.2	35038.3	100
evaluation	50	44775.9	58208.0	188.2[off request path

Confirmed misses: 50/50 · end_to_end = gateway span (ask() start to final received). Stage shares sum to 100% with gateway_overhead (consumer join, embedding, result logging). evaluation runs after the final answer, off the request path; its share is relative to the same end-to-end total for comparison only.

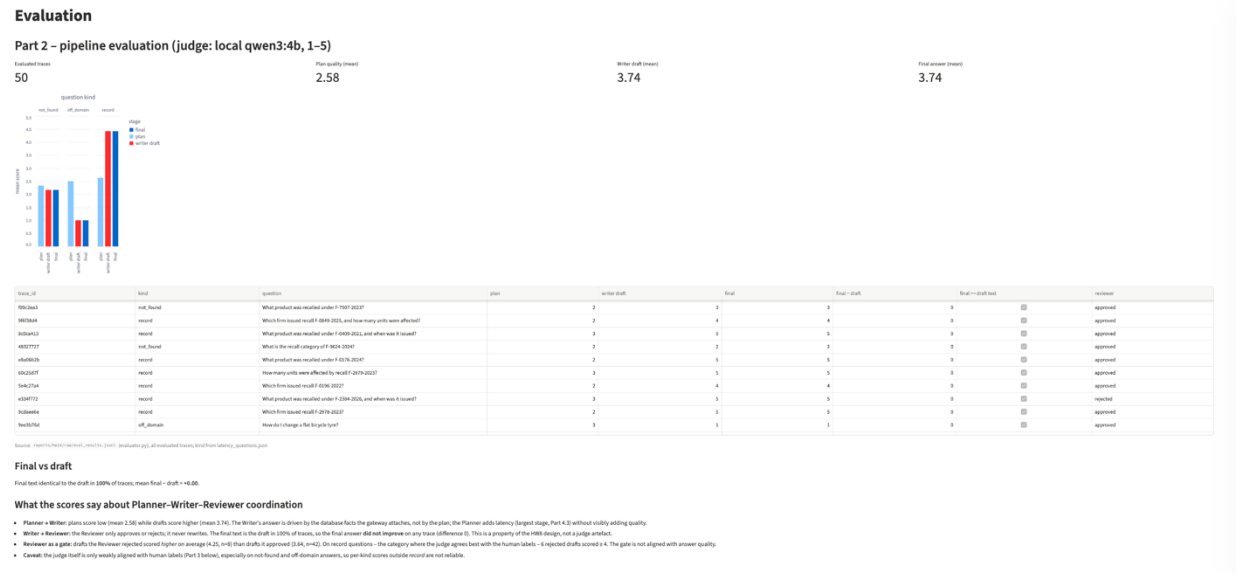
Source: reports/hw3/raw/latency_summary.json (latency_experiment.py analyze over spans.jsonl)

Recent requests

start	trace_id	cache	outcome	status	duration_ms	question	error
2026-11-13 11:01:46	3f2d1ca3-0b0d-46b9-9a15-c6bcd583b81e	hit	cached	ok	139.2	Which firm issued recall F-0002-20217	None
2026-11-13 11:01:37	42b9eaab-700d-4529-91af-67ba4b4a09e6	hit	cached	ok	134.6	Which firm issued recall F-0002-20217	None
2026-11-13 11:01:28	d874ef79-b229-43d5-8fa9-3233b64919a8	hit	cached	ok	163.6	Which firm issued recall F-0002-20217	None
2026-11-13 11:01:16	29420e6b-1f61-48d3-9591-1aa45d5bc7367	hit	cached	ok	151.7	Which firm issued recall F-0002-20217	None
2026-11-13 11:01:06	5f287cae-937d-4b52-b116-dcd96f8a3c49	hit	cached	ok	136	Which firm issued recall F-0002-20217	None
2026-11-13 11:00:55	696e368d-6b17-4369-ada2-17742414198	hit	cached	ok	134.4	Which firm issued recall F-0002-20217	None
2026-11-13 11:00:43	33d14253-3fb0-489f-a2f0-bdc737ae6c85	hit	cached	ok	134.5	Which firm issued recall F-0002-20217	None
2026-11-13 11:00:33	155ee496-c336-4081-bbf1-c51be2f6e0e7	hit	cached	ok	158.6	Which firm issued recall F-0002-20217	None
2026-11-13 11:00:22	d0c49786-3cbc-4724-a2b4-0136715e3269	hit	cached	ok	179	Which firm issued recall F-0002-20217	None
2026-11-13 10:05:33	fb34a683-8eec-471a-b121-0835b7f2349c	miss	error	error	62632.2	How many units were affected by recall F-1003-20227	TimeoutError: no answer for fb34a683-8eec-471a-b1

Source: reports/hw3/raw/spans.jsonl , stage = gateway (one span per request); window 2026-11-13 09:01 → 2026-11-13 11:01 (all recorded)

Screenshot 1.1 — Summary page, window “All recorded”: requests, hit rate (hits vs misses), error rate, latency for misses and hits, each with its source and window.



Evaluation

Part 2 – pipeline evaluation (judge: local qwen3:4b, 1–5)

Evaluated traces	Plan quality (mean)	Writer draft (mean)	Final answer (mean)
50	2.58	3.74	3.74

question kind
not_found | off_domain | record

stage
final
plan
writer draft

mean score

trace_id	kind	question	plan	writer draft	final	final − draft	final == draft text	reviewer
f0fc2ea3	not_found	What product was recalled under F-7597-20237	2	3	3	0	☑	approved
9f073bb4	record	Which firm issued recall F-0849-2023, and how many units were affected?	2	4	4	0	☑	approved
0c0be413	record	What product was recalled under F-0430-2022, and when was it issued?	3	3	3	0	☑	approved
48327727	not_found	What is the recall category of F-9424-20247	2	2	2	0	☑	approved
e8a68c2b	record	What product was recalled under F-0376-20247	2	5	5	0	☑	approved
60c2547f	record	How many units were affected by recall F-2079-20237	3	5	5	0	☑	approved
5e4c27a4	record	Which firm issued recall F-0196-20227	2	4	4	0	☑	approved
e3347772	record	What product was recalled under F-2394-2026, and when was it issued?	3	5	5	0	☑	rejected
0c8aeede	record	Which firm issued recall F-2079-20237	2	5	5	0	☑	approved
9ee3b76d	off_domain	How do I change a flat bicycle tyre?	3	1	1	0	☑	approved

Source: reports/hw3/raw/eval_results.json (evaluator.py), all evaluated traces; kind from latency_questions.json

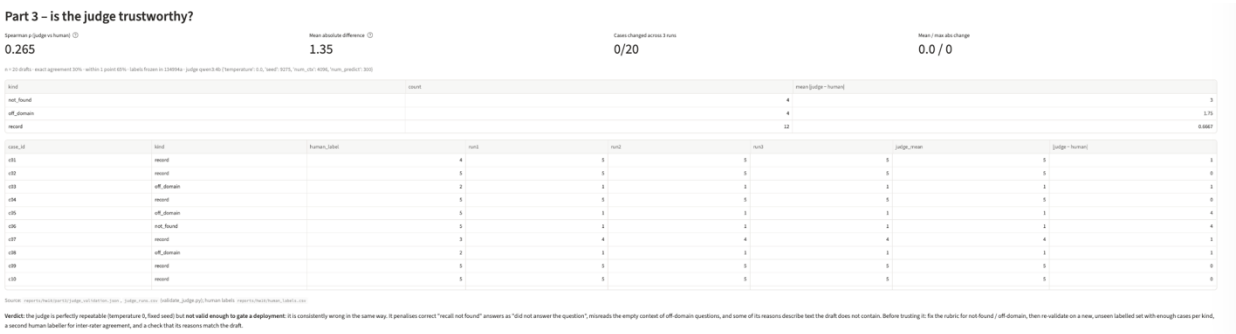
Final vs draft

Final text identical to the draft in **100%** of traces; mean final − draft = **+0.00**.

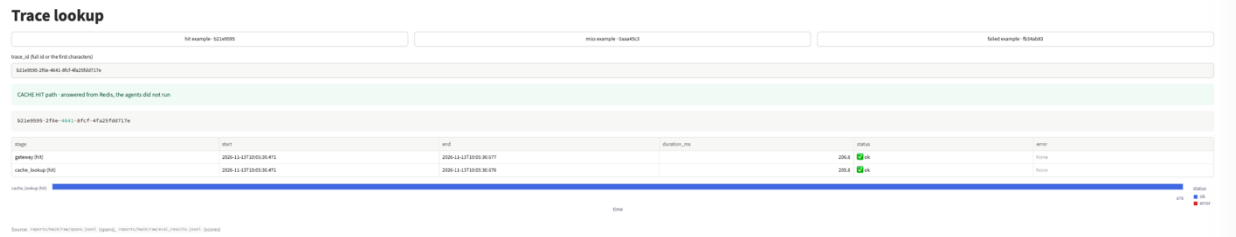
What the scores say about Planner–Writer–Reviewer coordination

- **Planner + Writer:** plans score low (mean 2.58) while drafts score higher (mean 3.74). The Writer's answer is driven by the database facts the gateway attaches, not by the plan; the Planner adds latency (largest stage, Part 4.3) without visibly adding quality.
- **Writer + Reviewer:** the Reviewer only approves or rejects; it never rewrites. The final text is the draft in 100% of traces, so the final answer **did not improve** on any trace (difference 0). This is a property of the HW8 design, not a judge artefact.
- **Reviewer as a gate:** drafts the Reviewer rejected scored higher on average (4.25, n=8) than drafts it approved (3.64, n=42). On record questions – the category where the judge agrees best with the human labels – 6 rejected drafts scored ≥ 4. The gate is not aligned with answer quality.
- **Caveat:** the judge itself is only weakly aligned with human labels (Part 3 below), especially on not-found and off-domain answers, so per-kind scores outside record are not reliable.

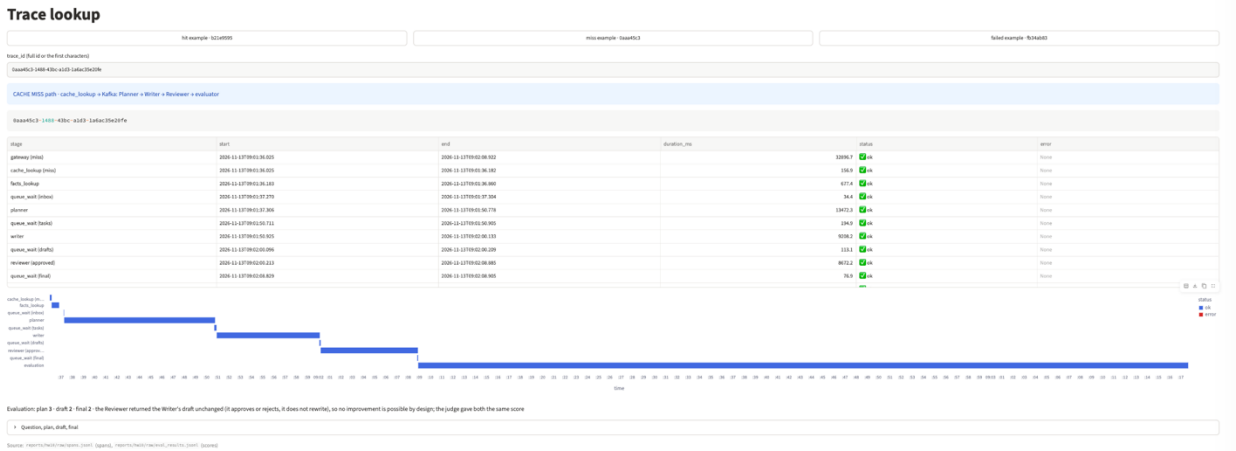
Screenshot 1.2a — Evaluation page, Part 2: mean scores by stage and question kind, per-case table, final vs draft, coordination interpretation.



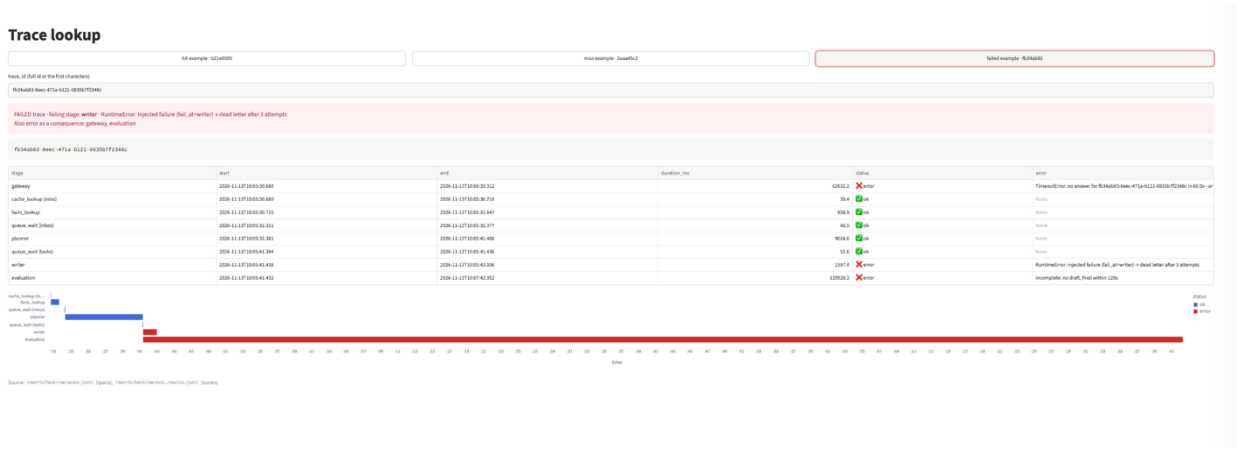
Screenshot 1.2b — Evaluation page, Part 3: Spearman ρ, mean absolute difference, repeatability, per-kind gap and raw per-case scores.



Screenshot 1.3a — Trace lookup — cache-hit example b21e9595: cache_lookup (hit) + gateway only.



Screenshot 1.3b — Trace lookup — cache-miss example 0aaa45c3: full journey in stage order with timeline.



Screenshot 1.3c — Trace lookup — failed example fb34ab83: failing stage writer, gateway and evaluation as consequences.

Part 2 — Automated Evaluation of the Kafka Pipeline

2.1 Evaluation inputs and linkage

`code/eval_pipeline/evaluator.py` is a separate consumer (group s9275_evaluator) on s9275_tasks (plan), s9275_drafts (draft) and s9275_final (final answer). It buffers messages by correlation_id and scores only when all three are present. Each result line in raw/eval_results.jsonl keeps the question, facts, plan, draft, final answer, Reviewer verdict and reason, the topic / partition / offset / timestamp of the three source messages, the judge settings, and every score with its reason.

code/eval_pipeline/evaluator.py — join on correlation_id

```
            entry = pending.setdefault(cid, {"first_seen": time.time(), "parts": {}})
            entry["parts"][ROLE[msg.topic()]] = {"message": message, "source": source(msg)}
            log(AGENT, "RECEIVED", f"cid={cid} {ROLE[msg.topic()]} "
                                   f"({len(entry['parts'])}/3)", cid=cid)
            if len(entry["parts"]) < 3:
                continue

            pending.pop(cid)
            log(AGENT, "JUDGING", f"cid={cid} 3 judge calls ...", cid=cid)
            record = evaluate(cid, entry["parts"], plan_m, help_m)
            save(record)
```

Bounded: (1) join timeout — a correlation_id still missing a message after 120 s is written as status “incomplete” with an error span; (2) every judge call has a 90 s HTTP timeout and num_predict 300, and a failed call is recorded for that metric only; (3) a message that is not JSON or has no correlation_id is logged and skipped; (4) the Kafka group join waits at most 30 s.

code/eval_pipeline/evaluator.py — join timeout

```
def expire(pending: dict, done: set) -> None:
    """Join timeout: give up on correlation_ids that are still missing a message."""
    now = time.time()
    for cid in [c for c, p in pending.items() if now - p["first_seen"] > JOIN_TIMEOUT_S]:
        p = pending.pop(cid)
        missing = [r for r in ("plan", "draft", "final") if r not in p["parts"]]
        write_span(cid, "evaluation", p["first_seen"], now, "error",
```

```
                f"incomplete: no {', '.join(missing)} within {JOIN_TIMEOUT_S:.0f}s")
        save({"trace_id": cid, "status": "incomplete", "missing": missing,
              "sources": {r: v["source"] for r, v in p["parts"].items()},
              "evaluated_at": datetime.now().isoformat(timespec="seconds")})
        log(AGENT, "INCOMPLETE", f"cid={cid} missing={missing}", cid=cid)
        done.add(cid)
```

Judge (local, documented): qwen3:4b via Ollama, called directly by a DeepEval model wrapper (LocalJudge). qwen3:8b was tried first: on 8 GB it ran 24% on CPU and a 5-token reply took 81.5 s (a JSON reply did not finish in 300 s). qwen3:4b runs 100% on GPU, about 12 s per score, and is a different model from the generator (qwen2.5:3b). The GEval rubric has five bands (1,1)…(5,5), so DeepEval asks the judge for a 1–5 score directly; evaluation_steps are written out, so each score is one model call.

code/eval_pipeline/judge.py — LocalJudge.generate

```
    def generate(self, prompt: str, schema=None):
        response = self.load_model().chat(
            model=self.name,
            messages=[{"role": "user", "content": prompt}],
            format=schema.model_json_schema() if schema else None,
            options=JUDGE_OPTIONS,
            think=False,
        )
        text = response.message.content
        return schema.model_validate_json(text) if schema else text
```

code/eval_pipeline/judge.py — 1–5 rubric (also used for the human labels)

```
HELPFULNESS_RUBRIC = [
    "Very poor: does not answer the question, or states values that contradict the "
    "database facts, or invents facts (numbers, names, dates, reasons).",
    "Poor: only partly answers; contains an error or an unsupported claim that could "
    "mislead a store employee.",
    "Acceptable: the requested value is correct, but the answer is vague, padded, or "
    "leaves out something the question asked for.",
    "Good: correct values taken from the facts and a direct answer; small issues such "
    "as extra wording or a minor irrelevant sentence.",
    "Excellent: answers exactly what was asked with the correct values, invents "
    "nothing, concise and immediately useful.",
]
```

2.2 Criteria and end-to-end result

Criterion	Judge input	Saved
Plan Quality	question + plan	score, reason
Writer Helpfulness	question + draft + database facts	score, reason, trace_id
Final Answer Helpfulness	question + final answer + database facts	score, reason, final answer
Final-vs-Draft Improvement	the two Helpfulness scores	draft score, final score, difference, identical-text flag, interpretation

End-to-end example (the cache-miss example trace `0aaa45c3`, “When was recall F-9616-2024 issued?”, a recall number that does not exist): plan 3 (“steps relevant but lack specificity…”), Writer draft 2 (“correctly states the recall does not exist… adds speculation about it being

‘unique or outdated’”), final 2, difference 0; the Reviewer approved the draft and passed it on unchanged.

All 50 latency requests (Part 4.3) were evaluated:

Kind	n	Reviewer approved	Plan	Writer draft	Final
record	38	32	2.63	4.42	4.42
not found	6	5	2.33	2.17	2.17
off-domain	6	5	2.50	1.00	1.00
all	50	42	2.58	3.74	3.74

Final vs draft: the final text was identical to the draft in 50/50 traces and the difference was 0 on every trace. The final answer did **not** improve.

What the scores say about coordination:

- **Planner → Writer:** plans score low (2.58) while drafts score higher (3.74). The Writer's answer comes from the database facts the gateway attaches, not from the plan; the Planner is also the slowest stage (40% of a miss, Part 4.3).
- **Writer → Reviewer:** the Reviewer approves or rejects but never rewrites (answer = draft in reviewer.py). No improvement is possible by this design — the zero difference is a property of the pipeline, not a judge artefact.
- **Reviewer as a gate:** drafts it rejected scored higher (mean 4.25, n = 8) than drafts it approved (3.64, n = 42); six rejected drafts scored 5 (for example the correct “F-0004-2022 was not found”). The gate is not aligned with answer quality.
- **Caveat:** Part 3 shows the judge is reliable only on record questions; the not-found and off-domain means above are mostly judge error.

```
Xu Wang | DATA 260 HW10 | PREFIX s9275 | reprint of saved evidence
(saved evaluator log, reports/hw10/raw/agent_events.jsonl, trace 0aaa45c3)

08:54:24.974 [evaluator] READY      listening on s9275_tasks, s9275_drafts, s9275_final (group s9275_evaluator) judge=qwen
3:4b join_timeout=120s -> /Users/Sunny/Documents/data260-9275/reports/hw10/raw/eval_results.jsonl
09:01:50.893 [evaluator] RECEIVED   cid=0aaa45c3-1488-43bc-a1d3-1a6ac35e20fe plan (1/3)
09:02:00.176 [evaluator] RECEIVED   cid=0aaa45c3-1488-43bc-a1d3-1a6ac35e20fe draft (2/3)
09:02:08.906 [evaluator] RECEIVED   cid=0aaa45c3-1488-43bc-a1d3-1a6ac35e20fe final (3/3)
09:02:08.911 [evaluator] JUDGING    cid=0aaa45c3-1488-43bc-a1d3-1a6ac35e20fe 3 judge calls ...
09:03:17.735 [evaluator] SCORED     cid=0aaa45c3-1488-43bc-a1d3-1a6ac35e20fe plan=3 draft=2 final=2 diff=0 status=evaluate
d

question: When was recall F-9616-2024 issued?
  plan_quality         3  The steps are relevant to the question but lack specificity in identifying the exact date format
 or source of
  writer_helpfulness   2  The response correctly states that recall F-9616-2024 does not exist in the database, matching t
he Context. Ho
  final_helpfulness    2  The response correctly states that recall F-9616-2024 does not exist in the database, matching t
he Context. Ho
  final_vs_draft       {'draft_score': 2, 'final_score': 2, 'difference': 0, 'final_identical_to_draft': True, 'interpreta
tion': "the Reviewer returned the Writer's draft unchanged (it approves or rejects, it does not rewrite), so no improvemen
t is possible by design; the judge gave both the same score"}
  sources              {'plan': {'topic': 's9275_tasks', 'partition': 1, 'offset': 65, 'kafka_ts_ms': 1794589310711}, 'dra
ft': {'topic': 's9275_drafts', 'partition': 0, 'offset': 74, 'kafka_ts_ms': 1794589320096}, 'final': {'topic': 's9275_fina
l', 'partition': 0, 'offset': 74, 'kafka_ts_ms': 1794589328829}}
```

Screenshot 2.1 — Evaluator terminal: RECEIVED plan (1/3), draft (2/3), final (3/3), JUDGING, SCORED plan / draft / final / diff for one correlation_id. reprinted from raw/agent_events.jsonl by code/show_hw10_evidence.py evaluator

Part 3 — Validate the Judge

3.1 Frozen evaluation cases

`reports/hw10/eval_cases.json`: 20 Writer drafts from the Part 4.3 run, selected without looking at judge scores — stratified by question kind and Reviewer verdict (6 approved + 6 rejected record questions, 4 not-found, 4 off-domain; random.Random(9275)). `reports/hw10/human_labels.csv`: my own helpfulness label 1–5 per draft, assigned from labeling_sheet.md (question, facts, draft and the same rubric as the judge; no scores). Both were committed in **134994a**, before validate_judge.py existed; the judge results are in 957ff97, and the labels were never edited (checked by verify_hw10.py). These drafts had been scored once in Part 2 as part of the pipeline evaluation; the three judge runs below are new.

Screenshot 3.1 — git log --oneline: 134994a (eval_cases.json + human_labels.csv) before 957ff97 (judge validation).

3.2 Judge scores vs human labels

Metric	Value
Spearman ρ (human vs judge)	0.265 (p = 0.2597, not significant)
Mean absolute difference	1.35 points
Judge − human, mean	-0.75 (the judge is harsher)
Exact agreement / within 1 point	30% / 65%

Raw per-case data (part3/judge_runs.csv; every reason is in part3/judge_reasons.jsonl):

Case	Kind	Human	Run 1	Run 2	Run 3
c01	record	4	5	5	5
c02	record	5	5	5	5
c03	off domain	2	1	1	1
c04	record	5	5	5	5
c05	off domain	5	1	1	1
c06	not found	5	1	1	1
c07	record	3	4	4	4
c08	off domain	2	1	1	1
c09	record	5	5	5	5
c10	record	5	5	5	5
c11	record	4	5	5	5
c12	not found	5	2	2	2
c13	not found	5	3	3	3
c14	record	5	5	5	5
c15	not found	5	2	2	2
c16	record	5	5	5	5

Case	Kind	Human	Run 1	Run 2	Run 3
c17	record	5	3	3	3
c18	record	2	4	4	4
c19	off domain	2	1	1	1
c20	record	4	5	5	5

Kind	n	Mean |judge − human|
record	12	0.67
off-domain	4	1.75
not found	4	3.00

Why they disagree, from the judge's own reasons:

- The lowest rubric band says “does not answer the question”. For a recall that does not exist the correct answer is “not found”, yet the judge applied band 1 (c06: “does not answer the question (which asks for the state)” → 1; my label 5), although evaluation step 4 says the opposite.
- Off-domain questions get the context “No database record was looked up”, which the judge read as “everything is unsupported” (c05: Python / JavaScript for “which language should I learn first” → “invents languages” → 1).
- Some reasons describe text that is not in the draft (c17: “includes extra details like product name and category” — the draft is one sentence with the firm and the state).
- 12 of my 20 labels are 5; many ties in the human scores also lower ρ.

```
Xu Wang | DATA 260 HW10 | PREFIX s9275 | reprint of saved evidence
(saved run, reports/hw10/part3/judge_runs.csv + judge_validation.json)

case  kind        human  run1  run2  run3
c01   record          4     5     5     5
c02   record          5     5     5     5
c03   off_domain      2     1     1     1
c04   record          5     5     5     5
c05   off_domain      5     1     1     1
c06   not_found       5     1     1     1
c07   record          3     4     4     4
c08   off_domain      2     1     1     1
c09   record          5     5     5     5
c10   record          5     5     5     5
c11   record          4     5     5     5
c12   not_found       5     2     2     2
c13   not_found       5     3     3     3
c14   record          5     5     5     5
c15   not_found       5     2     2     2
c16   record          5     5     5     5
c17   record          5     3     3     3
c18   record          2     4     4     4
c19   off_domain      2     1     1     1
c20   record          4     5     5     5

labels frozen in: 134994a HW10 Part 3.1: freeze 20 Writer drafts and hand-assigned helpfulness labels (before any judge validation run)
judge: {'model': 'qwen3:4b', 'options': {'temperature': 0.0, 'seed': 9275, 'num_ctx': 4096, 'num_predict': 300}, 'timeout_s': 90.0, 'metric': 'Helpfulness (judge.py)', 'run
s': 3}
{
  "vs_human (run1)": {
    "n": 20,
    "spearman_rho": 0.265,
    "spearman_p": 0.2597,
    "mean_abs_diff": 1.35,
    "mean_judge_minus_human": -0.75,
    "exact_agreement": 0.3,
    "within_1": 0.65
  },
  "repeatability": {
    "cases": 20,
    "cases_with_any_change": 0,
    "share_cases_changed": 0.0,
    "run_pairs_compared": 60,
    "share_pairs_changed": 0.0,
    "mean_abs_change": 0.0,
    "max_abs_change": 0,
    "changed_case_ids": [],
    "judge_errors": 0
  }
}
```

Screenshot 3.2 — validate_judge.py: labels-frozen commit, judge settings, and the final judge_validation.json (vs_human and repeatability). reprinted from part3/ by code/show_hw10_evidence.py judge

3.3 Repeatability

Three runs over the same 20 drafts, same order, same settings (temperature 0, seed 9275, num_ctx 4096, num_predict 300): **0 of 20** cases changed score, 60 of 60 run pairs identical, mean absolute change 0, maximum 0, 0 judge errors. The judge is deterministic, including its reasons.

Would I let this judge gate a deployment? No. It is perfectly repeatable but consistently wrong on two of the three answer kinds; repeatability without validity only repeats the error. Before trusting it I would require: (1) a rubric that states “not found” and a polite decline are correct answers, and a clearer context for off-domain questions; (2) a new labelled set, never used while changing the rubric, with about 20+ cases per kind and per-kind agreement reported; (3) a second human labeller, to know the human–human agreement the judge should reach; (4) a check that the judge's reasons match the draft; (5) a comparison with a stronger judge model. I did not change the rubric after seeing these results, because tuning it on the same 20 drafts would invalidate the validation.

Part 4 — Distributed Tracing and Latency Analysis

4.1 Trace propagation

trace_id is the HW8 correlation_id. It is now created once, at the start of SemanticCache.ask(), **before** the cache lookup (in HW9 it was created only after a miss, so a cache hit had no id), and on a miss it is sent as the Kafka correlation_id. No second request identifier exists; verify_hw10.py checks that every agent and evaluator span belongs to a request with a gateway span.

code/cache_app/semantic_cache.py — trace_id before the cache lookup

```
    def ask(self, question: str, trace_id: Optional[str] = None,
            extra: Optional[dict] = None) -> dict:
        """HW10: the trace_id is created here, BEFORE the cache lookup, so hits
        and misses both have one; on a miss it becomes the Kafka correlation_id."""
        trace_id = trace_id or new_trace_id()
        with span(trace_id, "gateway", question=question) as g:   # = end-to-end span
            out = self._ask(question, trace_id, extra)
            g["cache"] = "hit" if out["is_cached"] else "miss"
            # A Reviewer rejection is an answer outcome, not a system error: the request
            # completed. Errors are exceptions/timeouts (span() records those itself).
            g["outcome"] = "cached" if out["is_cached"] else out.get("status")
            return out
```

code/observability/tracing.py — one JSON span per stage

```
@contextmanager
def span(trace_id: str, stage: str, **extra: Any) -> Iterator[Dict[str, Any]]:
    """Time the block; on an exception record status=error with the message, then re-raise."""
    fields: Dict[str, Any] = dict(extra)
    start = time.time()
    try:
        yield fields
    except BaseException as exc:          # also Ctrl+C / SystemExit: an interrupted request
still leaves a span
        write_span(trace_id, stage, start, time.time(), "error",
                   f"{type(exc).__name__}: {exc}", **fields)
        raise
```

```
    write_span(trace_id, stage, start, time.time(), fields.pop("status", "ok"),
               fields.pop("error", None), **fields)


def record_queue_wait(trace_id: str, hop: str, msg: Any) -> Dict[str, Any]:
    """Queue wait for one Kafka hop = now - the producer's CreateTime stamped on the
message."""
    _, produced_ms = msg.timestamp()
    return write_span(trace_id, "queue_wait", produced_ms / 1000, time.time(), hop=hop)
```

code/kafka_agents/planner.py — queue wait + stage span (Writer / Reviewer the same)

```
            record_queue_wait(cid, "inbox", msg)            # HW10: hop 1, gateway -> Planner

            with span(cid, "planner"):                      # HW10: model call + send
                steps = parse_steps(llm(SYSTEM, message["question"], max_tokens=200))
                log(AGENT, "PLANNED", f"cid={cid} steps={steps}", cid=cid)

                out = {**message, "plan": steps, "trace": add_trace(message, AGENT)}
                send(producer, TASKS, out)
```

Stage	Written by	Fields beyond trace_id / start / end / duration / status / error
gateway	semantic_cache.py	whole request = end-to-end; cache; outcome (cached / approved / rejected)
cache_lookup	semantic_cache.py	cache = hit / miss; similarity; miss_reason
facts_lookup	semantic_cache.py	MySQL record for Writer / Reviewer (miss only)
queue_wait	each consumer	hop = inbox / tasks / drafts / final (receive time − Kafka CreateTime)
planner / writer / reviewer	the agents	writer = error when the message goes to the dead letter; review_status
evaluation	evaluator.py	judge; plan / draft / final scores; error on join timeout or judge failure

A Reviewer rejection is recorded as outcome = rejected, not as an error: the request completed. Errors are exceptions and timeouts.

4.2 Required trace examples (raw/trace_examples.json)

Cache hit — `b21e9595-2f0e-4641-8fcf-4fa25fdd717e` (“Which firm issued recall F-0002-2021?”):

Stage	Start	End	ms	Status	Error
gateway (hit)	10:05:30.471	10:05:30.677	206.8	ok	
cache_lookup (hit)	10:05:30.471	10:05:30.676	205.8	ok	

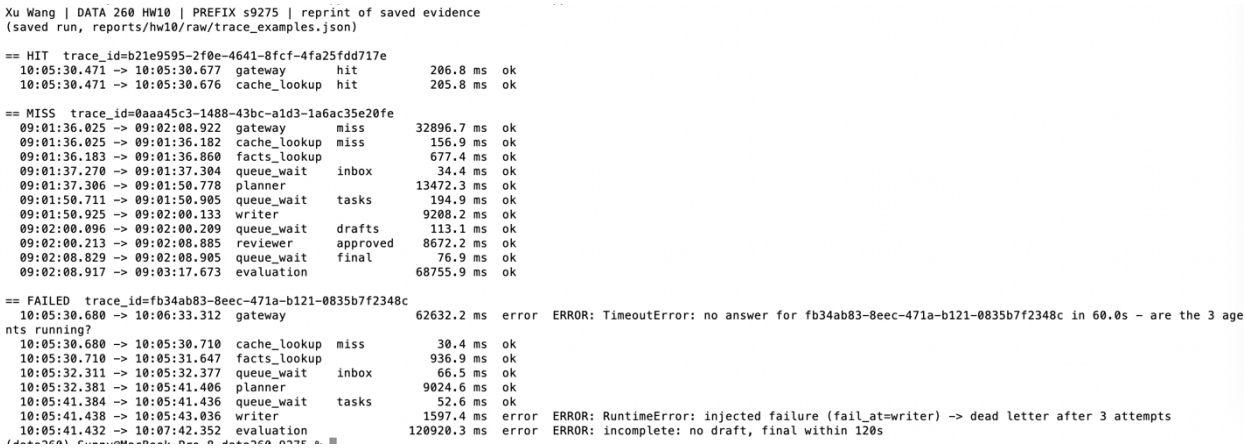
Cache miss, completed — `0aaa45c3-1488-43bc-a1d3-1a6ac35e20fe`:

Stage	Start	End	ms	Status	Error
gateway (miss)	09:01:36.025	09:02:08.922	32,896.7	ok	

Stage	Start	End	ms	Status	Error
cache_lookup (miss)	09:01:36.025	09:01:36.182	156.9	ok	
facts_lookup	09:01:36.183	09:01:36.860	677.4	ok	
queue_wait (inbox)	09:01:37.270	09:01:37.304	34.4	ok	
planner	09:01:37.306	09:01:50.778	13,472.3	ok	
queue_wait (tasks)	09:01:50.711	09:01:50.905	194.9	ok	
writer	09:01:50.925	09:02:00.133	9,208.2	ok	
queue_wait (drafts)	09:02:00.096	09:02:00.209	113.1	ok	
reviewer (approved)	09:02:00.213	09:02:08.885	8,672.2	ok	
queue_wait (final)	09:02:08.829	09:02:08.905	76.9	ok	
evaluation	09:02:08.917	09:03:17.673	68,755.9	ok	

Failed — `fb34ab83-8eec-471a-b121-0835b7f2348c` (sent with {"fail_at": "writer"}): failing stage **writer**; the gateway (60 s timeout) and the evaluator (no draft within 120 s) fail as a consequence.

Stage	Start	End	ms	Status	Error
gateway	10:05:30.680	10:06:33.312	62,632.2	error	TimeoutError: no answer for fb34ab83-8eec-471a-b121-0835b7f2348c in 60
cache_lookup (miss)	10:05:30.680	10:05:30.710	30.4	ok	
facts_lookup	10:05:30.710	10:05:31.647	936.9	ok	
queue_wait (inbox)	10:05:32.311	10:05:32.377	66.5	ok	
planner	10:05:32.381	10:05:41.406	9,024.6	ok	
queue_wait (tasks)	10:05:41.384	10:05:41.436	52.6	ok	
writer	10:05:41.438	10:05:43.036	1,597.4	error	RuntimeError: injected failure (fail_at=writer) -> dead letter after 3
evaluation	10:05:41.432	10:07:42.352	120,920.3	error	incomplete: no draft, final within 120s



Screenshot 4.2 — python code/eval_pipeline/trace_examples.py — the three journeys printed in stage order. reprinted from raw/trace_examples.json by code/show_hw10_evidence.py traces

4.3 Stage-latency experiment — 50 confirmed cache misses

Questions (raw/latency_questions.json, random.Random(9275)): 38 about real recall records whose number has no cache entry, 6 about numbers that do not exist, 6 off-domain; each was pre-checked against the cache (0 replaced). During the run CACHE_STORE = 0, so no answer was written to the cache. From the spans: **50 / 50 cache_lookup spans are misses** and all 50 are complete (4 queue hops + evaluation). Requests were sent one at a time, each after the previous evaluation finished, because on 8 GB the judge and the agents share the GPU.

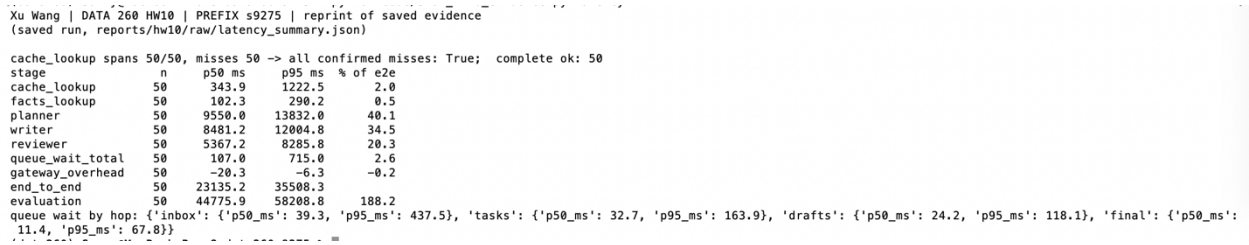
Stage	p50 (ms)	p95 (ms)	% of cache-miss end-to-end
Cache lookup	343.9	1,222.5	2
Facts lookup (MySQL)	102.3	290.2	0.5
Planner	9,550.0	13,832.0	40.1
Writer	8,481.2	12,004.8	34.5
Reviewer	5,367.2	8,285.8	20.3
Queue wait (total, 4 hops)	107.0	715.0	2.6
Gateway overhead	-20.3	-6.3	-0.2
End-to-end (cache miss)	23,135.2	35,508.3	100
Evaluation (off the request path)	44,775.9	58,208.8	(188.2)
Cache hit end-to-end (separate metric, n = 10)	145.5	194.3	–

Queue wait by hop, p50 / p95 ms: inbox 39.3 / 437.5 · tasks 32.7 / 163.9 · drafts 24.2 / 118.1 · final 11.4 / 67.8. Percentages = sum of the stage over the 50 requests ÷ sum of end-to-end. Gateway overhead (end-to-end − all stages) is slightly negative because each hop's queue wait starts at the producer's CreateTime while the sending stage's span still includes its flush(), so a few ms are counted twice.

- **Dominant contributor: the Planner (40.1%).** The three model stages together are 95%; queues and the cache are under 5%.

- The Planner took 5.8 s in a single test with the judge idle and 9.6 s p50 here: it is the first model call after the judge, so Ollama swaps qwen3:4b out and qwen2.5:3b in on this 8 GB Mac, and that load time lands in the Planner span.
- **What I would optimise first:** (1) move the judge off the agents' GPU (separate host, or batch evaluation outside serving hours) so no request pays a model swap; (2) replace the Planner's model call with a rule-based plan for the fixed lookup questions (units / firm / date / product / category of recall X), which are almost all of the traffic — Part 2 shows the plans add little quality.
- A cache hit costs 145 ms p50 (embedding + Redis KNN) against 23.1 s for a miss, about 160×; cache-hit latency is kept as a separate dashboard metric.

First attempt (discarded): two evaluator processes were in the s9275_evaluator group at once (a test instance left running), so Kafka split the partitions and one instance never received the plans (“incomplete [plan]”); the gateway consumer was also dropped after 300 s without poll() (MAXPOLL). Fix: exactly one process per consumer group (checked with kafka-consumer-groups --describe --members before the run) and max.poll.interval.ms = 1 h for the gateway consumer. All data was deleted and the run repeated with new trace_ids.



```
Xu Wang | DATA 260 HW10 | PREFIX s9275 | reprint of saved evidence
(saved run, reports/hw10/raw/latency_summary.json)

cache_lookup spans 50/50, misses 50 -> all confirmed misses: True;  complete ok: 50
stage                 n     p50 ms     p95 ms  % of e2e
cache_lookup         50      343.9     1222.5       2.0
facts_lookup         50      102.3      290.2       0.5
planner              50     9550.0    13832.0      40.1
writer               50     8481.2    12004.8      34.5
reviewer             50     5367.2     8285.8      20.3
queue_wait_total     50      107.0      715.0       2.6
gateway_overhead     50      -20.3       -6.3      -0.2
end_to_end           50    23135.2    35508.3
evaluation           50    44775.9    58208.8     188.2
queue wait by hop: {'inbox': {'p50_ms': 39.3, 'p95_ms': 437.5}, 'tasks': {'p50_ms': 32.7, 'p95_ms': 163.9}, 'drafts': {'p50_ms': 24.2, 'p95_ms': 118.1}, 'final': {'p50_ms':
 11.4, 'p95_ms': 67.8}}
```

Screenshot 4.3 — python code/eval_pipeline/latency_experiment.py analyze — 50/50 confirmed misses and the p50 / p95 / % table. reprinted from raw/latency_summary.json by code/show_hw10_evidence.py latency

Verification

`make verify-hw10` → reports/hw10/verification.json: **23 passed, 0 failed**. It needs nothing running: the tracing module and the three dashboard pages are executed for real, and everything that used Kafka or a model is recomputed from the saved evidence — label commit order, span schema, one identifier, 50 / 50 misses with the full path, the p50 / p95 table, the three trace examples, the four criteria with sources, the bounded evaluator, Spearman / MAD / repeatability from judge_runs.csv.

Screenshot V — make verify-hw10 — final lines with 23 passed, 0 failed.

```
===== verify-hw10 2026-11-13 11:07:19 =====
python -u verify_hw10.py 2>&1 | tee -a reports/hw10/RUN_LOG.txt
Xu Wang | DATA 260 HW10 self-check | 2026-11-13 11:07:19
[PASS] Repo: no uncommitted changes outside reports/  []
[PASS] Repo: eval_cases.json + human_labels.csv committed before the first Part 3 judge result  labels ['1
794594605', '134994a'], cases ['1794594605', '134994a'], judge ['1794595954', '957ff97']
[PASS] Repo: human_labels.csv never edited after it was committed  ['134994a']
[PASS] P4.1: every span has trace_id, stage, start, end, duration, status, error  578 spans
[PASS] P4.1: one identifier - every agent/evaluator span belongs to a request with a gateway span  61 requ
ests, orphans=[]
[PASS] P4.1: cache_lookup span on every request, hit/miss recorded
[PASS] P4.1: tracing.span() writes ok and error spans and re-raises the error  [('planner', 'ok'), ('write
r', 'error')]
[PASS] P4.3: 50 latency requests, 50 confirmed cache misses  50/50
[PASS] P4.3: each has the full path (4 queue hops + evaluation)  50
[PASS] P4.3: p50/p95 table recomputed from spans.jsonl  planner p50 9550.0 ms, 40.1% of e2e
[PASS] P4.2: hit trace = cache_lookup(hit) + gateway, no agent stage  ['cache_lookup', 'gateway']
[PASS] P4.2: miss trace completes the agent path, all ok  11
[PASS] P4.2: failed trace - writer error identified, gateway error  RuntimeError: injected failure (fail_a
t=writer) -> dead letter after 3
[PASS] P2.2: every evaluation has plan / draft / final scores on 1-5 with reasons  50
[PASS] P2.2: final-vs-draft saves both scores, difference = final - draft, interpretation
[PASS] P2.1: original texts and Kafka sources (topic/offset) kept for plan, draft, final
[PASS] P2.1: judge is a local model  {'model': 'qwen3:4b', 'options': {'temperature': 0.0, 'seed': 9275, '
num_ctx': 4096, 'num_predict': 300}, 'scale': '1-5'}
[PASS] P2.1: evaluator is bounded - the failed trace was closed as incomplete
[PASS] P3.1: 20 frozen cases and 20 human labels in 1..5
[PASS] P3.2: Spearman and MAD recomputed from judge_runs.csv  rho=0.265 MAD=1.35
[PASS] P3.3: 3 runs per draft, repeatability recomputed  0/20 changed
[PASS] P1: dashboard uses st.cache_data and st.cache_resource
2026-11-13 11:07:20.729 WARNING streamlit.runtime.scriptrunner_utils.script_run_context: Thread 'MainThrea
d': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-11-13 11:07:22.086 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when ru
nning in bare mode.
2026-11-13 11:07:22.380 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when ru
nning in bare mode.
2026-11-13 11:07:22.380 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when ru
nning in bare mode.
[PASS] P1: Summary, Evaluation and Trace pages run without exceptions  {'summary_page': [], 'evaluation_pa
ge': [], 'trace_page': []}

23 passed, 0 failed -> reports/hw10/verification.json
```

AI Use

1. What I used an AI assistant for, and what I did myself.

I used Claude (Anthropic) as a coding assistant.

For each part, Claude proposed a design (the trace_id created before the cache lookup and reused as the Kafka correlation_id, one JSON span per stage in a shared file, queue wait from the Kafka CreateTime, a separate evaluator consumer that joins plan / draft / final on correlation_id, a local judge with a 1-5 rubric, a serial 50-request miss experiment, a dashboard that only reads saved evidence); I reviewed each proposal and approved it before any code was written.

I first wrote the code file by file and drafted METRICS.md and this report; Claude reviewed every file and edited the report.

I chose to keep the judge rubric unchanged after the smoke test, hand-assigned the 20 helpfulness labels from the labelling sheet and committed them before the validation script existed, ran every command on my Mac (Kafka, Redis, MySQL, Ollama, the three agent terminals and the evaluator, the latency run, the trace examples, the judge validation, the dashboard and the self-check), took every screenshot, and checked each result before committing.

2. One AI-produced output that was wrong.

The HW9 gateway code (written by Claude) waited for Kafka without a limit. In `SemanticCache.run_agent()` the gateway consumer joined the `s9275_final` topic with `while not consumer.assignment(): consumer.poll(0.2)`. When Kafka is not running, no partition is ever assigned, so the request hangs forever - exactly what HW10 Part 2.1 says the system must not do.

3. How I detected it.

While testing the new trace_id on a cache hit, my test question turned out to be a stale cache entry (miss_reason "stale") and went to the agent path while Kafka was not started. The terminal printed `Connect to ipv4#127.0.0.1:9092 failed: Connection refused` without end and never returned. I pasted the output; the span file showed the cache_lookup and facts_lookup spans but no gateway span, because the request never finished.

4. What I changed and why it works now.

The join now has a 30 s deadline: if no partition is assigned in time the gateway closes the consumer and raises `ConnectionError`, and the gateway span records it as `status = error`. The evaluator got the same 30 s limit, and `tracing.span()` now catches `BaseException`, so even a Ctrl+C leaves an error span. Later, in the first 50-request run, the gateway consumer was dropped after 300 s without `poll()` (MAXPOLL) while the script waited for the evaluator; its `max.poll.interval.ms` is now 1 hour. The failed-trace example (Part 4.2) and the self-check show that a failure now ends with an error span instead of a hang.

A second AI error is reported in the results rather than fixed: the judge context Claude wrote for questions without a database record ("No database record was looked up") made the judge treat correct off-domain answers as invented (Part 3). I found it by reading the judge's reasons after validation; I did not change the judge, because tuning it on the same 20 labelled drafts would invalidate the validation.